Actividad Sesión No. 2 — ¿Cuál algoritmo escala mejor?

Estructura de Datos I · Ingeniería de Sistemas – Ingeniería de Software · III semestre
Estudiante: _____ Fecha: _____

1. Clasificación de los cinco fragmentos (sección 6)

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	Se consulta una sola posición del arreglo; el trabajo no depende del tamaño de los datos.
Recorrer un arreglo	n	O(n)	Un único ciclo visita cada elemento una vez; el trabajo crece proporcionalmente con n.
Dos ciclos anidados	$n \times n = n^2$	$O(n^2)$	Por cada una de las n iteraciones del ciclo externo, el interno repite n veces.
Dividir n entre 2 repetidamente	$\log_2 n$	O(log n)	En cada iteración el problema se reduce a la mitad, por lo que se llega a 1 en $\sim\log_2 n$ pasos.
Ciclo n + reducción logarítmica	$n \times \log_2 n$	O(n log n)	Un ciclo que se ejecuta n veces y que en cada iteración realiza una reducción a la mitad (≈ log n pasos), como en el Ejercicio 5.

Nota: si la reducción logarítmica se ejecutara después del ciclo (secuencial y no dentro de él), el total sería n + log n y se simplificaría a O(n).

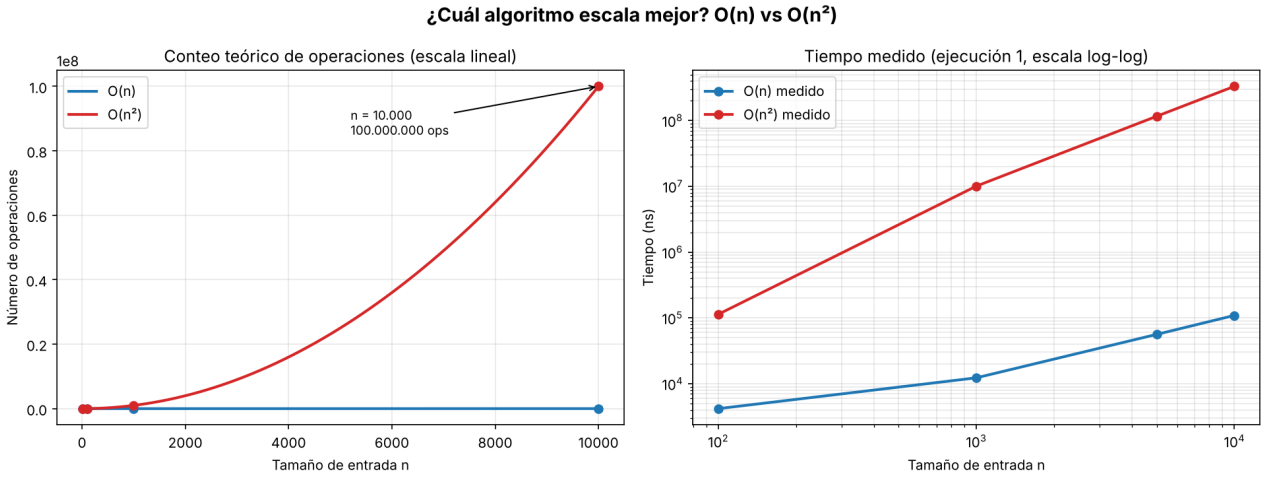
2. Tabla de tiempos (sección 10, ejecución 1)

Tamaño n	Tiempo O(n) (ns)	Tiempo $O(n^2)$ (ns)	¿Cuál fue más rápido?
100	4.182	113.829	O(n) — 27 veces más rápido
1.000	12.297	10.080.080	O(n) — 820 veces más rápido
5.000	56.246	116.184.835	O(n) — 2.066 veces más rápido
10.000	109.143	330.927.514	O(n) — 3.032 veces más rápido

Conteo teórico de operaciones (Parte D)

n	O(1)	O(log n) aprox.	O(n)	O(n log n) aprox.	$O(n^2)$
10	1	≈ 3.3	10	≈ 33	100
100	1	≈ 6.6	100	≈ 664	10.000
1.000	1	≈ 10.0	1.000	≈ 9.966	1.000.000
10.000	1	≈ 13.3	10.000	≈ 132.877	100.000.000

3. Gráfica comparativa



A la izquierda, el conteo teórico muestra que O(n) crece como una recta mientras que O(n²) se dispara: pasar de n = 1.000 a n = 10.000 multiplica por 10 las operaciones lineales, pero por 100 las cuadráticas (de 1.000.000 a 100.000.000). A la derecha, los tiempos medidos confirman la misma tendencia: con n = 10.000 el algoritmo cuadrático tardó unas 3.000 veces más que el lineal.

4. Respuestas a las preguntas de análisis

1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?
El tiempo de ejecución es una medida concreta (nanosegundos, segundos) que depende del equipo, el sistema operativo, la memoria y las optimizaciones de Java. Big O no mide segundos: describe cómo crece el número de operaciones cuando aumenta el tamaño de entrada n, es decir, la tendencia de crecimiento, independiente de la máquina.

2. ¿Por qué O(1) se considera una complejidad eficiente?
Porque el trabajo no cambia aunque crezca n. Acceder a datos[0] cuesta lo mismo con 10 elementos que con 1.000.000, así que escala perfectamente.

3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?
El número de operaciones (y aproximadamente el tiempo) se duplica: crece de forma proporcional. Si con n = 5.000 hace 5.000 pasos, con n = 10.000 hará 10.000.

4. ¿Qué ocurre aproximadamente con un algoritmo O(n²) cuando duplicamos n?
Las operaciones se multiplican por cuatro, porque $(2n)^2 = 4n^2$. Por ejemplo, de n = 5.000 (25.000.000 operaciones) a n = 10.000 (100.000.000 operaciones).

5. ¿Por qué los ciclos anidados suelen relacionarse con O(n²)?
Porque por cada iteración del ciclo externo (n veces) el ciclo interno se ejecuta completo (otras n veces), lo que da $n \times n = n^2$ operaciones. Si los ciclos fueran consecutivos, las operaciones se sumarían ($n + n = 2n$) y la complejidad seguiría siendo O(n).

6. ¿Qué característica permite reconocer un algoritmo O(log n)?
Que en cada paso el problema se reduce a una fracción (normalmente a la mitad), como en while (n > 1) { n = n / 2; }. El tamaño del problema no se recorre completo sino que se descarta una gran parte en cada iteración; es el patrón de la búsqueda binaria.

7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?
El algoritmo B, el cuadrático O(n²). En la ejecución 1 pasó de 113.829 ns (n = 100) a 330.927.514 ns (n = 10.000), unas 2.900 veces más, mientras que el lineal pasó de 4.182 ns a 109.143 ns, unas 26 veces más (con n 100 veces mayor).

8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?
No. Los tiempos cambian en cada ejecución porque dependen de los procesos que corren en el computador, la memoria disponible, el planificador del sistema operativo y las optimizaciones de la JVM. En las ejecuciones 2 y 3 incluso el tiempo cuadrático con n = 10.000 bajó a menos de 1 ms, porque el compilador JIT optimizó el ciclo (el contador no se usa, así que el trabajo se eliminó o aceleró). Esto demuestra que no se debe confundir el tiempo medido con la complejidad Big O: la complejidad teórica sigue siendo O(n²).

9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?
Preferiría O(log n), O(n) u O(n log n). Con n = 10.000.000, un algoritmo O(n) hace $\sim 10^7$ operaciones y uno O(n log n) $\sim 2{,}3 \times 10^8$, ambos viables en segundos. Uno O(n²) haría 10^{14} operaciones: a 10^9 operaciones por segundo tardaría unas 28 horas, lo que lo hace inviable.

10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?
Porque un programa que funciona con pocos datos puede degradarse al crecer el volumen de información, aumentando costos, tiempos de respuesta y consumo de recursos. Analizar la complejidad permite anticipar cómo escalará la solución y elegir una alternativa adecuada antes de que el problema aparezca en producción.

Pregunta orientadora

Elegiría el programa de 1.000 operaciones: resuelve el mismo problema con mil veces menos trabajo, y con millones de registros esa diferencia se traduce en un tiempo de respuesta mucho menor.

Reto adicional

Los dos ciclos for son consecutivos, no anidados, por lo que sus operaciones se suman: n + n = 2n. Al eliminar la constante, la complejidad es **O(n)**, no O(n^2).

5. Conclusión personal

Esta actividad me mostró que un programa correcto no siempre es un programa bueno. Al comparar el algoritmo lineal con el cuadrático vi que, con 100 datos, ambos terminan casi al instante, pero con 10.000 el cuadrático tardó miles de veces más. Eso explica por qué Big O describe la tendencia de crecimiento y no los segundos exactos. También aprendí que medir tiempo es poco confiable: los resultados cambiaron entre ejecuciones y la optimización de Java hizo que el algoritmo cuadrático pareciera más rápido de lo que realmente es. Por eso la medición experimental debe acompañarse del análisis teórico. Para una aplicación con millones de registros elegiría siempre una complejidad O(n) u O(n log n) y evitaría los ciclos anidados sobre todo el conjunto de datos. Como futura ingeniera o ingeniero de software, analizaré la eficiencia antes de decidir cómo resolver un problema.